Labsheet 2 : BTCS301P-AS03

Computer Vision Lab

Khushi Choudhary CU240251564

In [2]:
```
# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Set inline plotting size
plt.rcParams['figure.figsize'] = (10, 6)
```

In [3]:
```
# Generate a test image and corrupted variants
# Create a base grayscale image (300x300)
base_img = np.zeros((300, 300), dtype=np.uint8)
cv2.rectangle(base_img, (40, 40), (140, 140), 200, -1)      # Bright square
cv2.circle(base_img, (200, 200), 60, 255, -1)               # Bright circle
cv2.line(base_img, (30, 260), (270, 40), 150, 4)            # Diagonal edge

# Add Gaussian Noise
gauss_noise = np.random.normal(0, 25, base_img.shape).astype('int16')
gaussian_noisy_img = np.clip(base_img.astype('int16') + gauss_noise, 0, 255).ast

# Add Salt-and-Pepper Noise
sp_noisy_img = base_img.copy()
num_salt = int(0.02 * base_img.size)
num_pepper = int(0.02 * base_img.size)

# Salt (white pixels)
coords_salt = [np.random.randint(0, i - 1, num_salt) for i in base_img.shape]
sp_noisy_img[tuple(coords_salt)] = 255

# Pepper (black pixels)
coords_pepper = [np.random.randint(0, i - 1, num_pepper) for i in base_img.shape
sp_noisy_img[tuple(coords_pepper)] = 0

# Save locally to simulate loading external files
cv2.imwrite('clean_test.png', base_img)
cv2.imwrite('gaussian_noisy.png', gaussian_noisy_img)
cv2.imwrite('sp_noisy.png', sp_noisy_img)
```

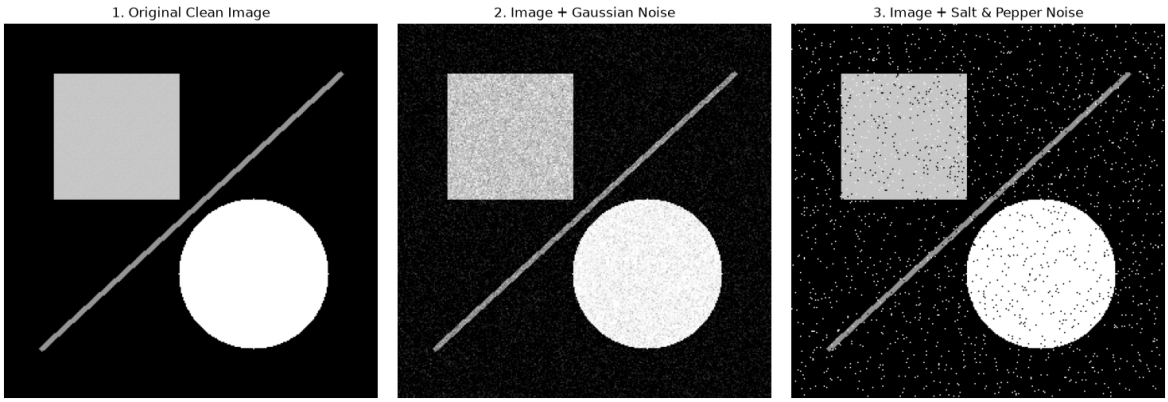
Out[3]: True

In [4]:
```
# Load images and inspect initial states
img_clean = cv2.imread('clean_test.png', cv2.IMREAD_GRAYSCALE)
img_gauss = cv2.imread('gaussian_noisy.png', cv2.IMREAD_GRAYSCALE)
img_sp = cv2.imread('sp_noisy.png', cv2.IMREAD_GRAYSCALE)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(img_clean, cmap='gray')
axes[0].set_title("1. Original Clean Image")
axes[1].imshow(img_gauss, cmap='gray')
axes[1].set_title("2. Image + Gaussian Noise")
axes[2].imshow(img_sp, cmap='gray')
```

```
axes[2].set_title("3. Image + Salt & Pepper Noise")

for ax in axes:
    ax.axis('off')
plt.tight_layout()
plt.show()
```



In [5]:
```
# Apply smoothing filters
kernel_size = 5
# 1. Mean (Average) Filter
mean_filtered = cv2.blur(img_gauss, (kernel_size, kernel_size))
# 2. Gaussian Filter
gaussian_filtered = cv2.GaussianBlur(img_gauss, (kernel_size, kernel_size), sigm
# 3. Median Filter (Applied to Salt-and-Pepper noise)
median_filtered = cv2.medianBlur(img_sp, kernel_size)

# Display Comparison
fig, axes = plt.subplots(2, 2, figsize=(10, 10))
axes[0, 0].imshow(img_gauss, cmap='gray')
axes[0, 0].set_title("Input: Gaussian Noise")

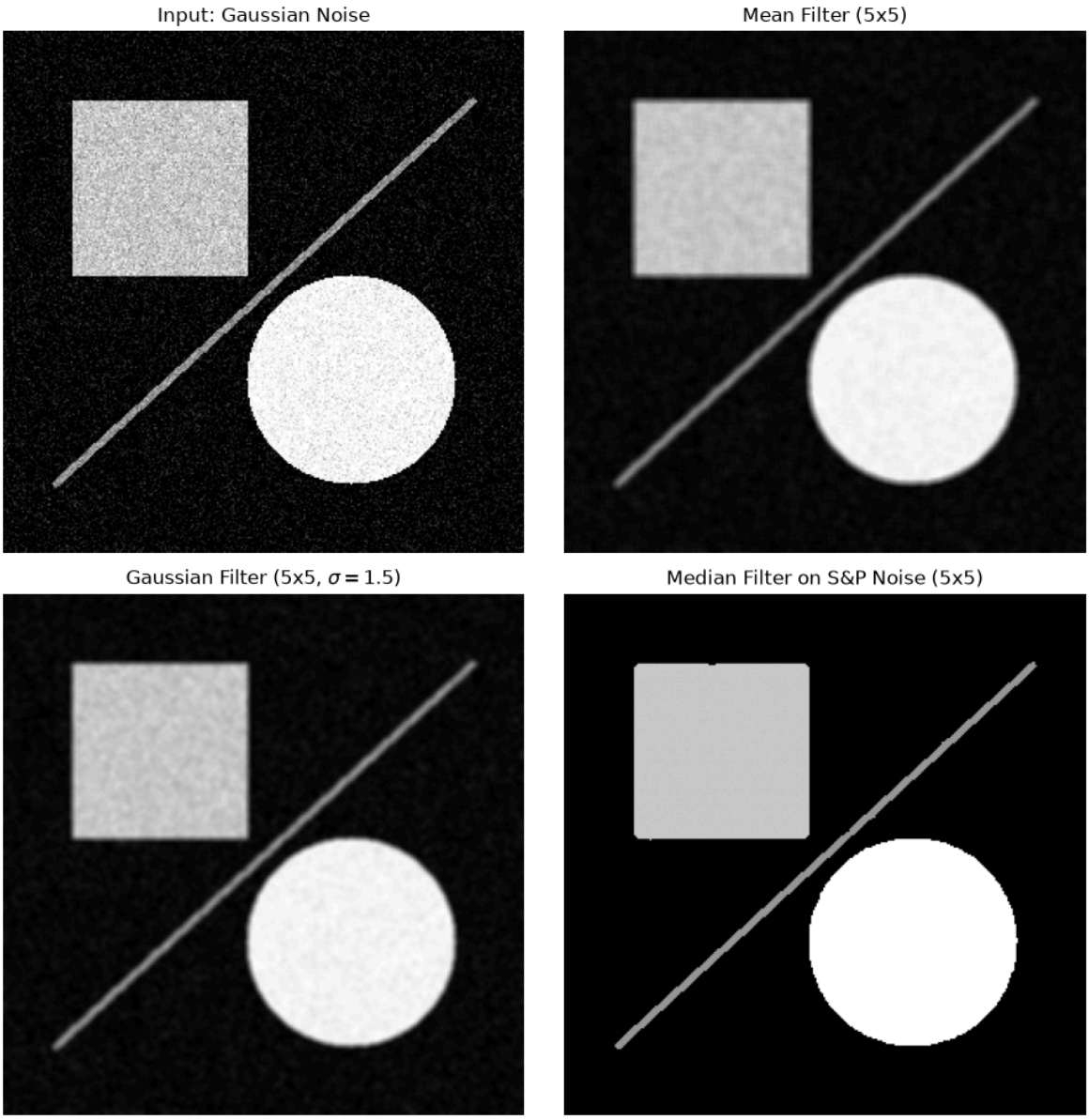
axes[0, 1].imshow(mean_filtered, cmap='gray')
axes[0, 1].set_title("Mean Filter (5x5)")

axes[1, 0].imshow(gaussian_filtered, cmap='gray')
axes[1, 0].set_title("Gaussian Filter (5x5, $\sigma=1.5$)")

axes[1, 1].imshow(median_filtered, cmap='gray')
axes[1, 1].set_title("Median Filter on S&P Noise (5x5)")

for ax in axes.ravel():
    ax.axis('off')
plt.tight_layout()
plt.show()
```

```
<>:19: SyntaxWarning: invalid escape sequence '\s'
<>:19: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_24300/28589700.py:19: SyntaxWarning: invalid escape sequence '\s'
  axes[1, 0].set_title("Gaussian Filter (5x5, $\sigma=1.5$)")
```



In [6]:
```
# Apply edge-enhancement and gradient operators
# 1. Laplacian Operator (2nd Derivative)
laplacian = cv2.Laplacian(img_clean, cv2.CV_64F, ksize=3)
laplacian_abs = np.uint8(np.absolute(laplacian))
# 2. Sobel Operators (1st Derivative)
sobel_x = cv2.Sobel(img_clean, cv2.CV_64F, 1, 0, ksize=3)  # Horizontal edges
sobel_y = cv2.Sobel(img_clean, cv2.CV_64F, 0, 1, ksize=3)  # Vertical edges

sobel_x_abs = np.uint8(np.absolute(sobel_x))
sobel_y_abs = np.uint8(np.absolute(sobel_y))

# Gradient Magnitude Combination: sqrt(Gx^2 + Gy^2)
sobel_combined = cv2.magnitude(sobel_x, sobel_y)
sobel_combined = np.uint8(np.clip(sobel_combined, 0, 255))
# Display Results
fig, axes = plt.subplots(2, 2, figsize=(10, 10))
axes[0, 0].imshow(laplacian_abs, cmap='gray')
axes[0, 0].set_title("Laplacian (2nd Derivative)")

axes[0, 1].imshow(sobel_combined, cmap='gray')
axes[0, 1].set_title("Sobel Magnitude ($\sqrt{G_x^2 + G_y^2}$)")

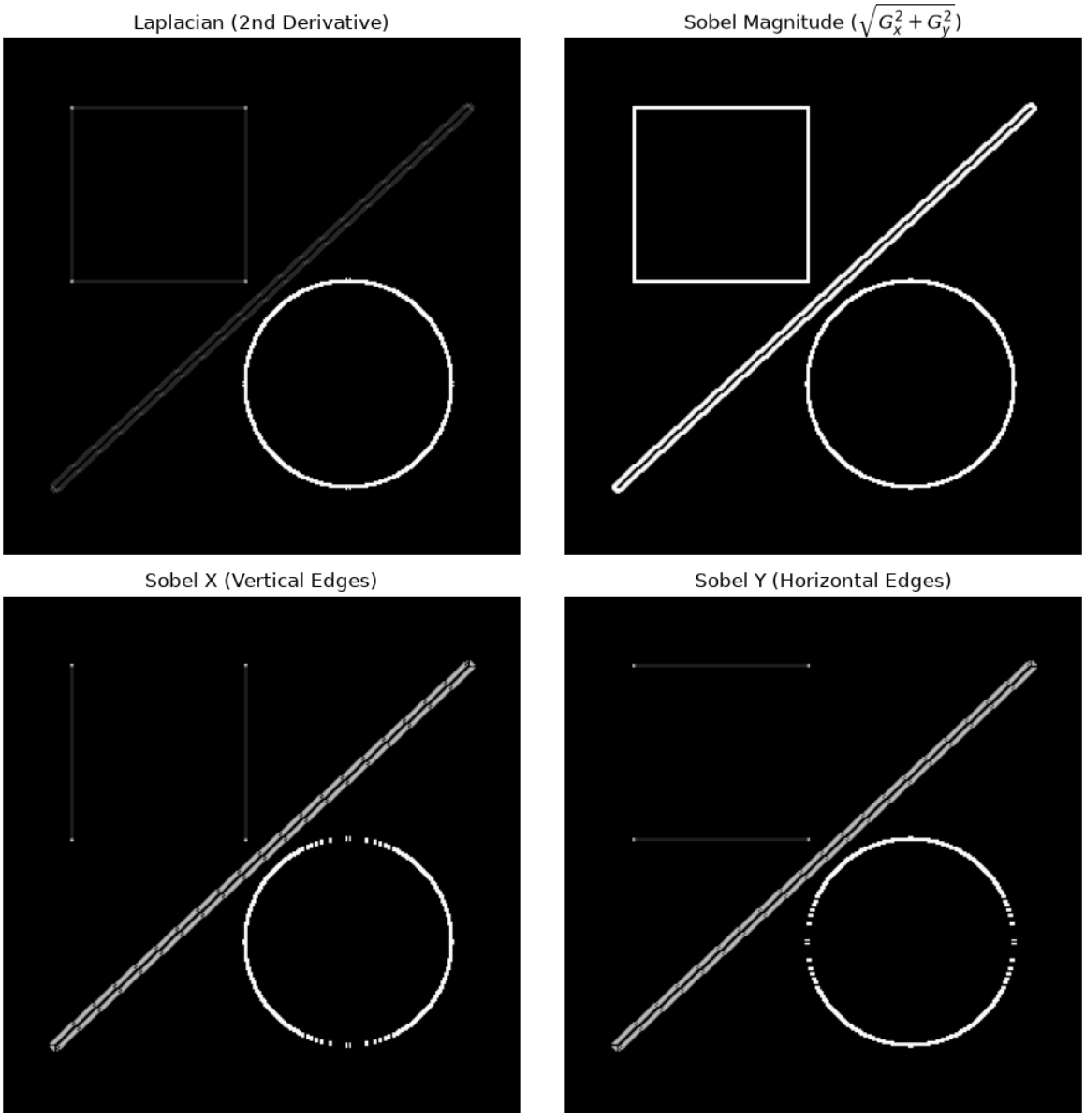
axes[1, 0].imshow(sobel_x_abs, cmap='gray')
```

```
axes[1, 0].set_title("Sobel X (Vertical Edges)")

axes[1, 1].imshow(sobel_y_abs, cmap='gray')
axes[1, 1].set_title("Sobel Y (Horizontal Edges)")

for ax in axes.ravel():
    ax.axis('off')
plt.tight_layout()
plt.show()
```

```
<>:21: SyntaxWarning: invalid escape sequence '\s'
<>:21: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_24300/1519766077.py:21: SyntaxWarning: invalid escape sequence
'\s'
  axes[0, 1].set_title("Sobel Magnitude ($\sqrt{G_x^2 + G_y^2}$)")
```



In [7]:
```
# Measure performance (PSNR and Execution Time)
import time
def compute_psnr(original, processed):
    mse = np.mean((original.astype(np.float64) - processed.astype(np.float64)) *
    if mse == 0:
        return float('inf')
    max_pixel = 255.0
    return 20 * np.log10(max_pixel / np.sqrt(mse))
```

```python
filters = {
    "Mean Filter": lambda img: cv2.blur(img, (5, 5)),
    "Gaussian Filter": lambda img: cv2.GaussianBlur(img, (5, 5), 1.5),
    "Median Filter": lambda img: cv2.medianBlur(img, 5),
    "Laplacian": lambda img: cv2.Laplacian(img, cv2.CV_64F),
    "Sobel": lambda img: cv2.Sobel(img, cv2.CV_64F, 1, 1)
}

print(f"{'Filter':<18} | {'Execution Time (ms)':<20} | {'PSNR vs Clean (dB)':<18
print("-" * 62)

for name, func in filters.items():
    start = time.perf_counter()
    res = func(img_gauss)
    elapsed = (time.perf_counter() - start) * 1000
    psnr_val = compute_psnr(img_clean, np.uint8(np.absolute(res)))
    print(f"{name:<18} | {elapsed:<20.4f} | {psnr_val:<18.2f}")
```

```
Filter             | Execution Time (ms)  | PSNR vs Clean (dB)
--------------------------------------------------------------
Mean Filter        | 0.1047               | 22.80
Gaussian Filter    | 0.1634               | 23.55
Median Filter      | 0.3208               | 31.44
Laplacian          | 0.1924               | 7.79
Sobel              | 0.1649               | 7.78
```